



Name: Rahin Ibne Harun

Id: 23-54177-3

Sec: U

Course: Compiler Design Assignment

Final Lab Task -3

Task 1:

Task 1: Write a C++ program that takes a regular expression as input and first checks whether it is **valid** or **invalid**.

The regular expression may contain:

- lowercase letters a to z as symbols
- union operator |
- Kleene star *
- plus operator +
- parentheses (and)

If the expression is valid, your program must:

- insert explicit concatenation operator . where needed
- convert the expression from **infix** form to **postfix** form

If the expression is invalid, print: **Invalid Expression**

Rules

- * means zero or more repetition
- + means one or more repetition
- brackets must be balanced
- expression cannot start with |, *, or +
- expression cannot end with |

Sample Input

a(b|c)*d

Sample Output

Valid Expression

Normalized: a.(b|c)*.d

Postfix: abc|*.d.

Code:

```
#include <iostream>

#include <string>

#include <stack>

using namespace std;

bool isLetter(char c) {
    return (c >= 'a' && c <= 'z');
}

bool isValid(string s) {
    if (s.empty()) return false;

    if (s[0] == '|' || s[0] == '*' || s[0] == '+') return false;
    if (s.back() == '|') return false;
```

```

int brackets = 0;
for (int i = 0; i < s.length(); i++) {
    if (s[i] == '(') brackets++;
    if (s[i] == ')') brackets--;
    if (brackets < 0) return false;

    if (i > 0) {
        char p = s[i-1];
        char c = s[i];
        if (c == '|' && p == '|') return false;
        if ((c == '*' || c == '+') && (p == '|' || p == '(')) return false;
    }
}
return brackets == 0;
}

```

```

string addDots(string s) {
    string res = "";
    for (int i = 0; i < s.length(); i++) {
        res += s[i];
        if (i + 1 < s.length()) {
            char curr = s[i];
            char next = s[i+1];

            bool currentEndsOperand = (isLetter(curr) || curr == '*' || curr == '+' || curr == '(');
            bool nextStartsOperand = (isLetter(next) || next == '(');

            if (currentEndsOperand && nextStartsOperand) {
                res += '.';
            }
        }
    }
}

```

```
    return res;
}
```

```
int getPriority(char c) {
    if (c == '*' || c == '+') return 3;
    if (c == '.') return 2;
    if (c == '|') return 1;
    return 0;
}
```

```
string toPostfix(string s) {
    string output = "";
    stack<char> st;

    for (char c : s) {
        if (isLetter(c)) {
            output += c;
        } else if (c == '(') {
            st.push(c);
        } else if (c == ')') {
            while (!st.empty() && st.top() != '(') {
                output += st.top();
                st.pop();
            }
            st.pop();
        } else {
            while (!st.empty() && st.top() != '(' && getPriority(st.top()) >= getPriority(c)) {
                output += st.top();
                st.pop();
            }
            st.push(c);
        }
    }
}
```

```
while (!st.empty()) {
    output += st.top();
    st.pop();
}
return output;
}

int main() {
    string input;
    cout << "Enter Expression: ";
    cin >> input;

    if (!isValid(input)) {
        cout << "Invalid Expression" << endl;
    } else {
        string dotted = addDots(input);
        string postfix = toPostfix(dotted);

        cout << "Valid Expression" << endl;
        cout << "Normalized: " << dotted << endl;
        cout << "Postfix: " << postfix << endl;
    }
    return 0;
}
)
```

Output:

```
"E:\Downloads\Compiler Final" × + v
Enter Expression: a(b|c)*d
Valid Expression
Normalized: a.(b|c)*.d
Postfix: abc|*.d.

Process returned 0 (0x0)    execution time : 30.358 s
Press any key to continue.
```

Task 2:

Task 2: Write a C++ program that takes the description of an **ϵ -NFA** and converts it into its equivalent **DFA** using the **subset construction method**.

Use **e** to represent epsilon transition.

Your program must:

- compute the **epsilon closure**
- generate all reachable DFA states as **sets of NFA states**
- print the **DFA transition table**
- clearly mark the **start state** and **final states**

After constructing the DFA, your program must also take multiple input strings and print whether each string is **Accepted** or **Rejected** by the generated DFA.

Input format

- First line: two integers n and m
n = number of states, numbered from 0 to n-1
m = number of input symbols
- Next m lines: the input symbols
- Next line: integer t, number of transitions
- Next t lines: current_state input_symbol next_state
- Next line: start state
- Next line: integer f, number of final states
- Next line: f integers representing final states
- Next line: integer q, number of test strings
- Next q lines: one input string per line

Output

Print:

- the DFA start state
- the DFA final states
- the complete DFA transition table
- then for each string:
 - Accepted
 - or Rejected

Sample Input	Sample Output
4 2 a b 6 0 e 1 0 e 2 1 a 1 1 b 3 2 a 3 3 b 3 0 1 3 4 a aa ab bbb	Start State = {0,1,2} Final States: {1,3} {3} Transition Table: {0,1,2} -a-> {1,3} {0,1,2} -b-> {3} {1,3} -a-> {1} {1,3} -b-> {3} {3} -a-> {} {3} -b-> {3} {1} -a-> {1} {1} -b-> {3} { } -a-> {} { } -b-> {} String: a Accepted

Code:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <set>
```

```
#include <map>
```

```
#include <queue>
```

```
#include <string>
```

```
using namespace std;
```

```
typedef set<int> StateSet;
```

```

void printSet(StateSet s) {
    if (s.empty()) {
        cout << "{}";
        return;
    }
    cout << "{";
    auto it = s.begin();
    while (it != s.end()) {
        cout << *it;
        if (++it != s.end()) cout << ",";
    }
    cout << "}";
}

StateSet getEClosure(StateSet states, int n, vector<pair<int, pair<char, int>>>& trans) {
    StateSet closure = states;
    bool changed = true;
    while (changed) {
        changed = false;
        for (auto& t : trans) {
            int u = t.first;
            char symbol = t.second.first;
            int v = t.second.second;
            if (symbol == 'e' && closure.count(u) && !closure.count(v)) {
                closure.insert(v);
                changed = true;
            }
        }
    }
    return closure;
}

StateSet getMove(StateSet states, char symbol, vector<pair<int, pair<char, int>>>& trans) {
    StateSet res;

```

```

for (int s : states) {
    for (auto& t : trans) {
        if (t.first == s && t.second.first == symbol) {
            res.insert(t.second.second);
        }
    }
}
return res;
}

```

```

int main() {
    int n, m;
    cin >> n >> m;
    vector<char> alphabet(m);
    for (int i = 0; i < m; i++) cin >> alphabet[i];

    int t_count;
    cin >> t_count;
    vector<pair<int, pair<char, int>>> transitions;
    for (int i = 0; i < t_count; i++) {
        int u, v; char c;
        cin >> u >> c >> v;
        transitions.push_back({u, {c, v}});
    }
}

```

```

int startNode; cin >> startNode;
int f_count; cin >> f_count;
set<int> nfaFinals;
for (int i = 0; i < f_count; i++) {
    int f; cin >> f;
    nfaFinals.insert(f);
}

```

```

map<StateSet, map<char, StateSet>> dfaTable;
vector<StateSet> dfaStates;
queue<StateSet> q;

StateSet startSet = getEClosure({startNode}, n, transitions);
q.push(startSet);
dfaStates.push_back(startSet);

int head = 0;
while(head < dfaStates.size()){
    StateSet curr = dfaStates[head++];
    for(char sym : alphabet) {
        StateSet next = getEClosure(getMove(curr, sym, transitions), n, transitions);
        dfaTable[curr][sym] = next;
    }
    bool found = false;
    for(auto& s : dfaStates) if(s == next) found = true;
    if(!found) dfaStates.push_back(next);
}

cout << "Start State = "; printSet(startSet); cout << endl;

cout << "Final States: ";
for(auto& s : dfaStates) {
    for(int state : s) {
        if(nfaFinals.count(state)) {
            printSet(s); cout << " ";
            break;
        }
    }
}

cout << "\nTransition Table:\n";
for(auto& s : dfaStates) {

```

```

    for(char sym : alphabet) {
        printSet(s); cout << " --" << sym << "--> ";
        printSet(dfaTable[s][sym]); cout << endl;
    }
}

int queries; cin >> queries;

while(queries--) {
    string str; cin >> str;
    StateSet curr = startSet;
    for(char c : str) curr = dfaTable[curr][c];

    bool accepted = false;
    for(int s : curr) if(nfaFinals.count(s)) accepted = true;
    cout << "String: " << str << " " << (accepted ? "Accepted" : "Rejected") << endl;
}

return 0;
}

```

Output:

4 2

a

b

6

0 e 1

0 e 2

1 a 1

1 b 3

2 a 3

3 a 3

0

1

3

Start State = {0,1,2}

Final States: {1,3} {3}

Transition Table:

{0,1,2} --a--> {1,3}

{0,1,2} --b--> {3}

{1,3} --a--> {1,3}

{1,3} --b--> {3}

{3} --a--> {3}

{3} --b--> {}

{ } --a--> { }

{ } --b--> { }

4

a

String: a Accepted

aa

String: aa Accepted

ab

String: ab Accepted

bbb

String: bbb Rejected

Process returned 0 (0x0) execution time : 76.497 s

Press any key to continue.

////////////////////////////////

End////////////////////////////////